

IHM - Introduction à Swing et AWT

API Swing :

Swing est la boîte à outils d'interface utilisateur de Java. Il a été développé durant l'existence de Java 1.1 et fait désormais partie des API centrales de Java 1.2 et supérieure.

Swing fournit des classes pour représenter des éléments d'interface comme les fenêtres, des boutons, des boîtes combo, des arborescences, des grilles et des menus - toute chose nécessaire à la construction d'une interface utilisateur dans une application Java. Pour cela, vous disposez du paquetage javax.swing (ainsi que ses nombreux sous-paquetages).

API AWT :

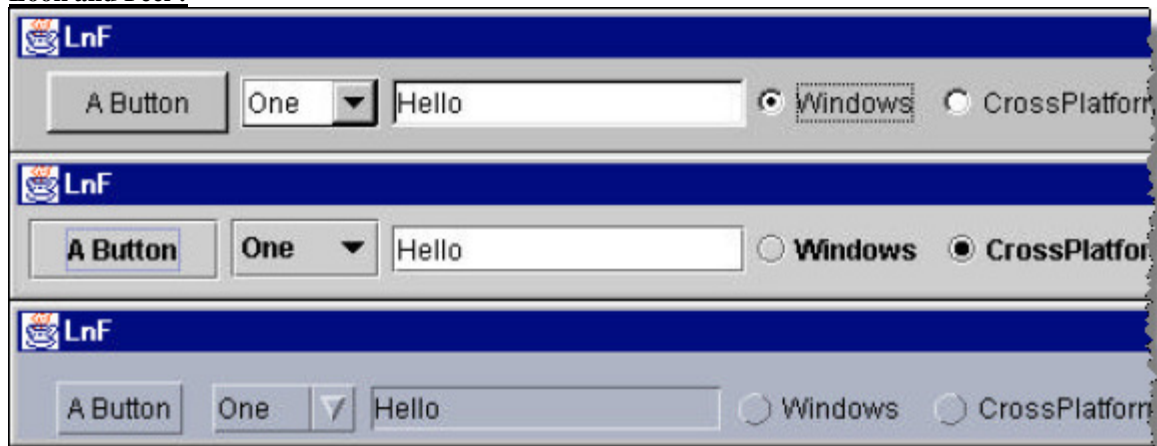
Pour comprendre Swing, il est utile de comprendre son prédécesseur AWT (Abstract Window Toolkit) issu du paquetage java.awt. Comme son nom l'indique, AWT est une abstraction. Comme le reste de java, il a été conçu pour être portable ; ses fonctionnalités sont les mêmes pour toutes les implémentations Java. Bien que les gens s'attendent généralement à ce que leurs applications aient un look-and-feel cohérent, il est souvent différent d'une plateforme à l'autre. C'est pourquoi, AWT a été conçu pour fonctionner de la même façon sur toutes les plate-formes, avec l'aspect du système natif.

Swing fournit également une API puissante de look-and-feel modifiable, qui permet de remplacer l'apparence du système d'exploitation natif au niveau de Java.

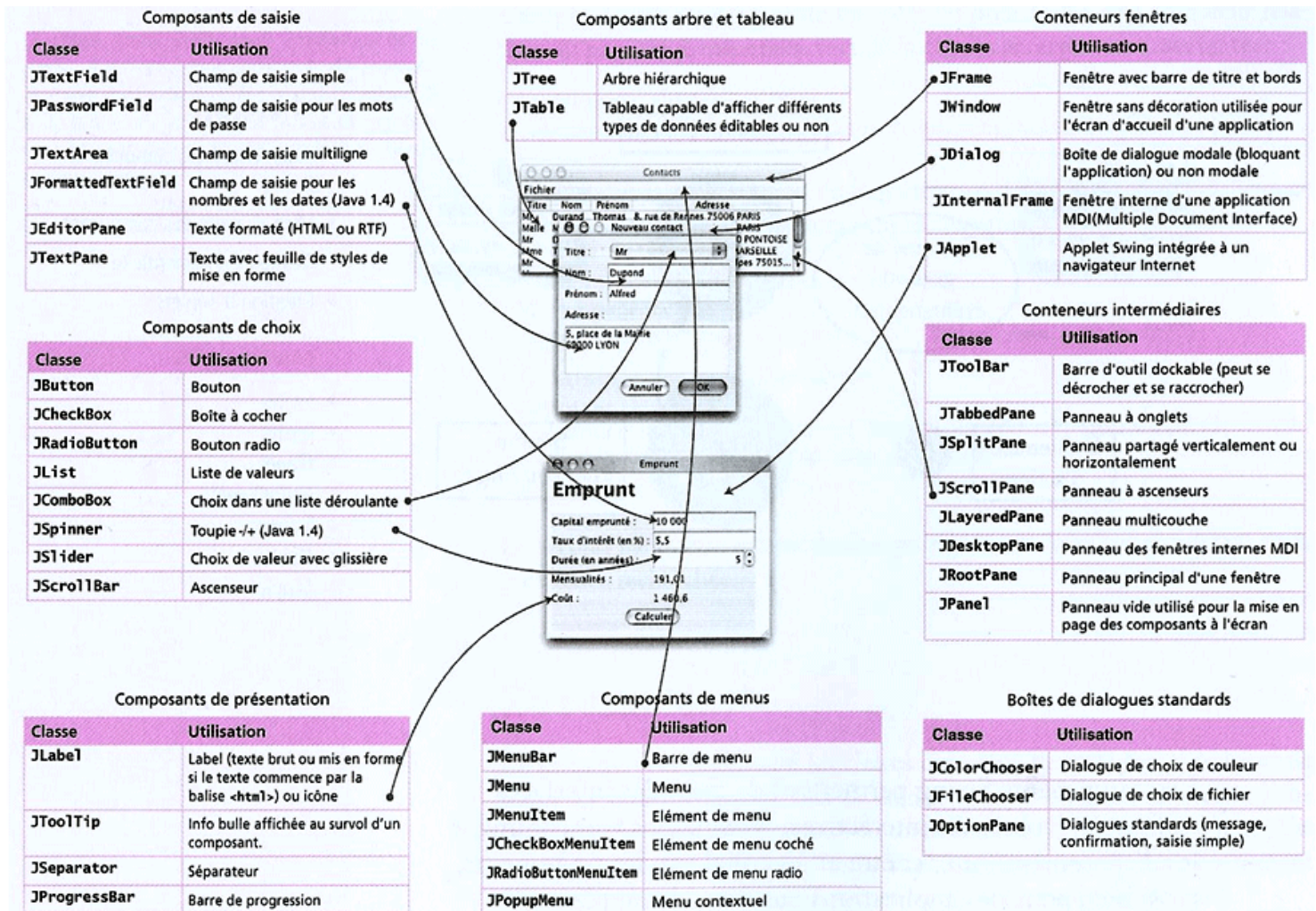
Il sera préférable de choisir l'interface utilisateur Swing. Toutefois, nous utiliserons certains des éléments d'AWT comme la classe Color et la gestion des événements, ainsi que quelques autres.

La plupart des classes de composant Swing commencent par la lettre J : JButton, JFrame, etc. Ce sont des classes comme Button et Frame, mais il s'agit dans ce dernier cas, comme nous l'avons vu, de composants AWT. Pour retrouver ces composants Swing, vous devez importer le paquetage javax.swing. En réalité JFrame hérite de Frame alors que les autres composants comme JButton, JLabel, JTextField, JMenu, etc. héritent tous de la classe de base JComponent qui fait partie de Swing, qui hérite lui-même indirectement de Component qui est un élément de AWT.

Look and Feel :



Les Composants:



Exemples de certains constructeurs:

Voir documentations de chaque composante dans ce lien:

<https://docs.oracle.com/javase/7/docs/api/allclasses-noframe.html>

Constructor Summary

Constructors

Constructor and Description

<code>JFrame()</code>

Constructs a new frame that is initially invisible.

<code>JFrame(GraphicsConfiguration gc)</code>

Creates a Frame in the specified GraphicsConfiguration of a screen device and a blank title.
--

<code>JFrame(String title)</code>

Creates a new, initially invisible Frame with the specified title.
--

<code>JFrame(String title, GraphicsConfiguration gc)</code>

Creates a JFrame with the specified title and the specified GraphicsConfiguration of a screen device.

Constructor Summary

Constructors

Constructor and Description

<code>JButton()</code>

Creates a button with no set text or icon.
--

<code>JButton(Action a)</code>

Creates a button where properties are taken from the Action supplied.

<code>JButton(Icon icon)</code>

Creates a button with an icon.

<code>JButton(String text)</code>

Creates a button with text.

<code>JButton(String text, Icon icon)</code>
--

Creates a button with initial text and an icon.

Constructor Summary

Constructors

Constructor and Description

<code>JLabel()</code>

Creates a JLabel instance with no image and with an empty string for the title.

<code>JLabel(Icon image)</code>

Creates a JLabel instance with the specified image.

<code>JLabel(Icon image, int horizontalAlignment)</code>
--

Creates a JLabel instance with the specified image and horizontal alignment.
--

<code>JLabel(String text)</code>

Creates a JLabel instance with the specified text.
--

<code>JLabel(String text, Icon icon, int horizontalAlignment)</code>
--

Creates a JLabel instance with the specified text, image, and horizontal alignment.

<code>JLabel(String text, int horizontalAlignment)</code>

Creates a JLabel instance with the specified text and horizontal alignment.

Constructor Summary

Constructors

Constructor and Description

`JList()`

Constructs a `JList` with an empty, read-only, model.

`JList(E[] listData)`

Constructs a `JList` that displays the elements in the specified array.

`JList(ListModel<E> dataModel)`

Constructs a `JList` that displays elements from the specified, non-null, model.

`JList(Vector<? extends E> listData)`

Constructs a `JList` that displays the elements in the specified `Vector`.

Constructor Summary

Constructors

Constructor and Description

`JCheckBox()`

Creates an initially unselected check box button with no text, no icon.

`JCheckBox(Action a)`

Creates a check box where properties are taken from the `Action` supplied.

`JCheckBox(Icon icon)`

Creates an initially unselected check box with an icon.

`JCheckBox(Icon icon, boolean selected)`

Creates a check box with an icon and specifies whether or not it is initially selected.

`JCheckBox(String text)`

Creates an initially unselected check box with text.

`JCheckBox(String text, boolean selected)`

Creates a check box with text and specifies whether or not it is initially selected.

`JCheckBox(String text, Icon icon)`

Creates an initially unselected check box with the specified text and icon.

`JCheckBox(String text, Icon icon, boolean selected)`

Creates a check box with text and icon, and specifies whether or not it is initially selected.

Exercices :

Solution1

```
import javax.swing.* ;
import java.awt.* ;
class MaFenetre extends JFrame{
    public MaFenetre (){
        super ("Ma premiere fenetre") ;
        setSize (300, 150) ;
        Container c = getContentPane() ;
        String[] lettres = {"a", "b", "c", "d", "e", "f", "g", "h", "i", "j" } ;
        JList liste = new JList(lettres) ;
```

```

        c.add(liste);
    }
}
public class Premfen{
    public static void main (String args[]){
        MaFenetre fen = new MaFenetre() ;
        fen.setVisible (true);
    }
}

```

Solution 2:

```

import javax.swing.* ;
import java.awt.*;
class MaFenetre extends JFrame{
    public MaFenetre (){
        super ("Ma premiere fenetre") ;
        setSize (300, 150) ;
        Container c = getContentPane() ;
        DefaultListModel modele = new DefaultListModel();
        modele.addElement("a");
        modele.addElement("b");
        modele.addElement("c");
        JList liste = new JList(modele) ;
        modele.addElement("d");
        c.add(liste);
    }
}
public class Premfen{
    public static void main (String args[]){
        MaFenetre fen = new MaFenetre() ;
        fen.setVisible (true);
    }
}

```

➔ **Avantage :** avec un DefaultListModel, on peut ajouter et enlever des éléments à une liste.

Quelques méthodes
void setSelectedIndex(int index) sélectionne l'élément numéro <i>index</i> (0 est le premier élément)
void setSelectionMode (int mode) - Si mode est égal à <code>ListSelectionModel.SINGLE_SELECTION</code> , alors on ne peut sélectionner qu'une seule valeur à la fois - Si mode est égal à <code>ListSelectionModel.SINGLE_INTERVAL_SELECTION</code> , alors on peut sélectionner une suite de valeurs successives - Si mode est égal à <code>ListSelectionModel.MULTIPLE_INTERVAL_SELECTION</code> , alors on peut sélectionner plusieurs valeurs à la fois n'importe où dans la liste (c'est celle par défaut)
Object getSelectedValue() Retourne l'élément sélectionné. Si cet élément est en réalité une chaîne de caractère, il faut utiliser la conversion explicite pour convertir un <i>Object</i> en un <i>String</i> (voir exemple ci-dessous)
Object[] getSelectedValues() Retourne un tableau d'objets contenant les éléments sélectionnés.
int getSelectedIndex() Retourne l'indice de l'élément sélectionné.
int[] getSelectedIndices() Retourne les indices des éléments sélectionnés.

Les gestionnaires de mise en forme :

Un gestionnaire de placement est un objet contrôlant le placement et le dimensionnement des composants situés à l'intérieur de la zone d'affichage d'un conteneur. Il ressemble au gestionnaire de fenêtres d'un système d'affichage ; il gouverne l'emplacement et la taille des composants. Chaque conteneur possède un gestionnaire de placement par défaut, mais il est possible d'en installer un nouveau en appelant sa méthode `setLayout()`.

Swing possède plusieurs gestionnaires de placement qui implémentent des modèles de disposition courants :

Classe	Description
<code>java.awt.FlowLayout</code>	Dispose les composants d'un container les uns derrière les autres en ligne et à leur taille préférée, en retournant à la ligne si le container n'est pas assez large.
<code>java.awt.GridLayout</code>	Dispose les composants d'un container dans une grille dont toutes les cellules ont les mêmes dimensions.
<code>java.awt.BorderLayout</code>	Dispose cinq composants maximum dans un container, deux au bords supérieur et inférieur à leur hauteur préférée, deux au bords gauche et droit à leur largeur préférée, et un au centre qui occupe le reste de l'espace.
<code>java.awt.CardLayout</code>	Affiche un composant à la fois parmi l'ensemble des composants d'un container (pratique pour créer des panneaux comme ceux de la boîte de dialogue de <i>Préférences</i> d'Eclipse).
<code>javax.swing.BoxLayout</code>	Dispose les composants en ligne à leur hauteur préférée ou en colonne à leur largeur préférée.
<code>java.awt.GridBagLayout</code>	Dispose les composants d'un container dans une grille dont les cellules peuvent avoir des dimensions variables. La position et les dimensions de la cellule d'un composant varient en fonction de sa taille préférée et des contraintes de classe <code>java.awt.GridBagConstraints</code> qui lui sont associées.
<code>javax.swing.SpringLayout</code>	Dispose les composants d'un container en fonction de leur taille préférée et de contraintes qui spécifient comment ces composants sont rattachés les uns par rapport aux autres.

```

package gestionnaire;

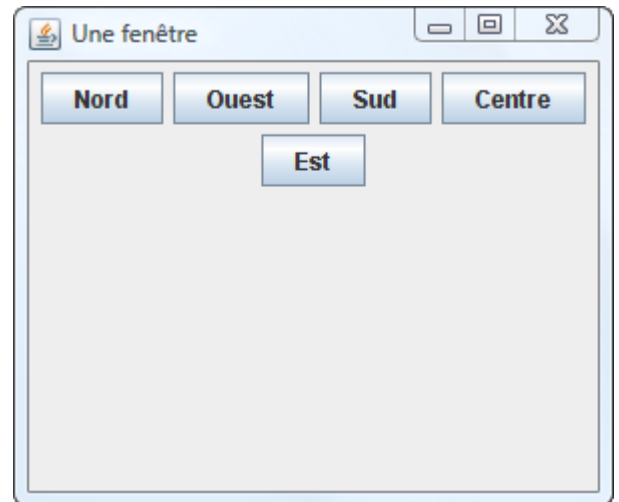
import javax.swing.*;

public class Fenêtre extends JFrame {
    private JButton nord = new JButton("Nord");
    private JButton ouest = new JButton("Ouest");
    private JButton sud = new JButton("Sud");
    private JButton centre = new JButton("Centre");
    private JButton est = new JButton("Est");
    private JPanel panneau = new JPanel();

    public Fenêtre() {
        setTitle("Une fenêtre");
        setSize(300, 250);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        panneau.add(nord);
        panneau.add(ouest);
        panneau.add(sud);
        panneau.add(centre);
        panneau.add(est);
        add(panneau);
        setVisible(true);
    }

    public static void main(String[] args) {
        new Fenêtre();
    }
}

```



```

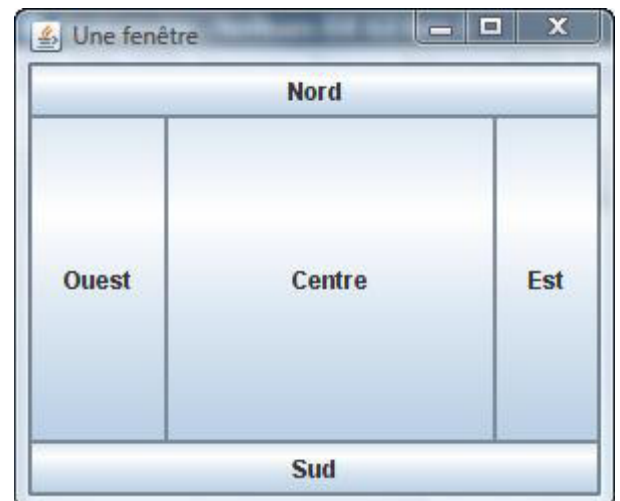
package gestionnaire;
import java.awt.BorderLayout;
import javax.swing.*;

public class Fenêtre extends JFrame {
    private JButton nord = new JButton("Nord");
    private JButton ouest = new JButton("Ouest");
    private JButton sud = new JButton("Sud");
    private JButton centre = new JButton("Centre");
    private JButton est = new JButton("Est");

    public Fenêtre() {
        setTitle("Une fenêtre");
        setSize(300, 250);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        add(nord, BorderLayout.NORTH);
        add(ouest, BorderLayout.WEST);
        add(sud, BorderLayout.SOUTH);
        add(centre, BorderLayout.CENTER);
        add(est, BorderLayout.EAST);
        setVisible(true);
    }

    public static void main(String[] args) {
        new Fenêtre();
    }
}

```



```

package gestionnaire;

```

```
import java.awt.*;
import javax.swing.*;
```

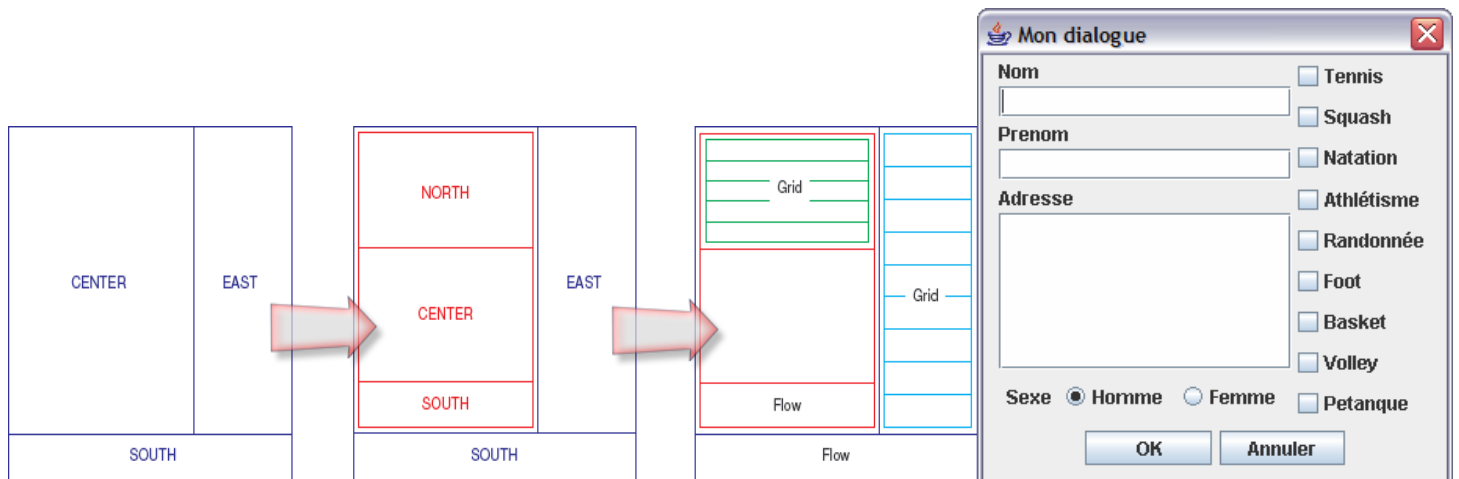
```
public class Fenêtre extends JFrame {
    private JButton nord = new JButton("Nord");
    private JButton ouest = new JButton("Ouest");
    private JButton sud = new JButton("Sud");
    private JButton centre = new
    JButton("Centre");
    private JButton est = new JButton("Est");

    public Fenêtre() {
        setTitle("Une fenêtre");
        setSize(300, 250);
    }
}
```

```
setDefaultCloseOperation(EXIT_ON_CLOSE);
setLayout(new GridLayout(3, 2));
add(nord);
add(ouest);
add(sud);
add(centre);
add(est);
setVisible(true);
}

public static void main(String[] args) {
    new Fenêtre();
}
}
```

L'idéal est de combiner l'ensemble de ces gestionnaires au travers de panneaux intermédiaires (JPanel) afin d'obtenir l'apparence désirée :



Autres exemples :

```
import javax.swing.*;
import java.awt.*;

class MaFenetre extends JFrame{
    public MaFenetre (){
        super ("Ma premiere fenetre") ;
        setSize (300, 300) ;

        setDefaultCloseOperation(EXIT_ON_CLOSE);
        Container c = getContentPane() ;

        Box bo = Box.createVerticalBox() ;
        c.add(bo) ;

        JLabel l1 ;
        l1 = new JLabel ("Saisir nom") ;
        bo.add(l1);
    }
}
```

```
JTextField t1 ;
t1 = new JTextField ("") ;
bo.add(t1);

JButton b1 ;
b1 = new JButton ("Valider") ;
bo.add(b1);
}

public class Premfen{
    public static void main (String args[]){
        MaFenetre fen = new MaFenetre();
        fen.setVisible (true);
    }
}
```


Remarques :

Le méthode `getPreferredSize()` indique la taille souhaitée mais pas celle toujours respectée. Ceci dépend du gestionnaire de mise en forme comme le montre le tableau suivant :

Layout	Hauteur	Largeur
Sans Layout	☒	☒
FlowLayout	☒	☒
BorderLayout(East, West)		☒
BorderLayout(North, South)	☒	
BorderLayout(Center)		
GridLayout		

Remarque : on peut utiliser la méthode **`pack()`** de `JFrame` pour redimensionner la fenêtre afin qu'elles tiennent compte de la taille préférée de ses différentes composantes.

Les événements :

I. Implémentation de l'interface `MouseListener`

La gestion d'un événement met en jeu un objet *source* et un objet *écouteur*.

L'objet qui a déclenché un événement est appelé *source*. Pour l'événement clique sur une fenêtre par exemple, la source est la fenêtre.

Pour traiter un événement, on associe à la source un *écouteur*. Cet *écouteur* doit être un objet dont la classe implémente une interface particulière correspondant à une *catégorie d'événements*. Il existe ainsi une interface correspondant à la *catégorie d'événement mouse* (événements souris), une interface correspondant à la *catégorie d'événement key* (événements clavier), une interface correspondant à la *catégorie d'événement action* (événement lié aux boutons et à d'autres composants). Chaque méthode appartenant à ce type d'interface représente un événement de la catégorie d'événement. Par exemple, l'interface correspondant la *catégorie d'événement souris* s'appelle `MouseListener` et elle contient 5 méthodes correspondant chacun à un événement de cette catégorie : la méthode `mousePressed`, la méthode `mouseReleased`, la méthode `mouseEntered`, la méthode `mouseExited` et la méthode `mouseClicked`.

1) Gestion de clic de souris

Supposons qu'on veut gérer l'événement clic de souris sur la fenêtre. On propose d'utiliser 4 méthodes :
1ère méthode : créer une classe qui implémente l'interface `MouseListener` et associer un objet de cette classe à la fenêtre. Cet objet représente l'écouteur de la fenêtre. Pour cela, on suivra les étapes suivantes :

Etape 1 : créer une classe qui implémente l'interface `MouseListener`. Les objets instanciés à partir de cette classe joueront le rôle d'écouteur. On appellera cette classe par exemple `EcouteurClasse`.

```
class EcouteurClasse implements MouseListener
{
    public void mouseClicked(MouseEvent ev) {
        int x = ev.getX(); // ( retourne l'abscisse du curseur au moment du clic
        int y = ev.getY(); // ( retourne l'ordonnée du curseur au moment du clic
        System.out.println ("clic au point de coordonnees " + x + ", " + y);
    };
    public void mousePressed(MouseEvent ev) {}
    public void mouseReleased(MouseEvent ev) {}
    public void mouseEntered(MouseEvent ev) {}
    public void mouseExited(MouseEvent ev) {}
}
```

Même si on a besoin que de la méthode `mouseClicked`, on doit implémenter toutes les méthodes de l'interface `MouseListener` (voir chapitre Héritage). On peut alors juste ouvrir une accolade et fermer une accolade pour faire l'implémentation des méthodes dont on n'a pas besoin (voir la méthode `mousePressed` par exemple).

Etape 2 : associer un écouteur de type `EcouteurClasse` à la fenêtre sur laquelle on va cliquer. Ceci se fait grâce à la méthode `addMouseListener` comme suit :

```
import javax.swing.*;
import java.awt.event.*;
class MaFenetre extends JFrame{
    public MaFenetre (){
        super ("Ma premiere fenetre");
        setSize (800, 120);
        addMouseListener(new EcouteurClasse());
    }
}
```

```

public class Premfen{
public static void main (String args[]){
MaFenetre fen = new MaFenetre();
fen.setVisible (true);
}
}

```

2^{ème} méthode : la fenêtre peut être elle-même son propre écouteur. Dans ce cas la fenêtre implémentera l'interface *MouseListener*, puis on associe l'objet courant (la fenêtre courante) à la fenêtre pour la désigner comme son propre écouteur. Pour cela, on suivra les étapes suivantes :

Etape 1 : changer la classe qui représente la fenêtre pour qu'elle implémente l'interface

MouseListener

```

import javax.swing.*;
import java.awt.event.*;
class MaFenetre extends JFrame implements MouseListener{
public MaFenetre (){
super ("Ma premiere fenetre") ;
setSize (800, 120) ;
}
public void mouseClicked (MouseEvent ev) {
int x = ev.getX();// □ retourne l'abscisse du curseur au moement du clic
int y = ev.getY();// □ retourne l'ordonnée du curseur au moement du clic
System.out.println ("clic au point de coordonnees " + x + ", " + y) ;
};
public void mousePressed (MouseEvent ev) {}
public void mouseReleased(MouseEvent ev) {}
public void mouseEntered (MouseEvent ev) {}
public void mouseExited (MouseEvent ev) {}
}

```

Etape 2 : considérer la fenêtre comme son propre écouteur :

```

import javax.swing.*;
import java.awt.event.*;
class MaFenetre extends JFrame implements MouseListener{
public MaFenetre (){
super ("Ma premiere fenetre") ;
setSize (800, 120) ;
addMouseListener(this);
}
public void mouseClicked (MouseEvent ev) {
int x = ev.getX();// □ retourne l'abscisse du curseur au moement du clic
int y = ev.getY();// □ retourne l'ordonnée du curseur au moement du clic
System.out.println ("clic au point de coordonnees " + x + ", " + y) ;
};
public void mousePressed (MouseEvent ev) {}
public void mouseReleased(MouseEvent ev) {}
public void mouseEntered (MouseEvent ev) {}
public void mouseExited (MouseEvent ev) {}
}
public class Premfen{
public static void main (String args[]){
MaFenetre fen = new MaFenetre();
fen.setVisible (true);
}
}

```

3^{ème} méthode: utiliser une classe prédéfinie appelée *MouseAdapter* qui implémente déjà l'interface *MouseListener*. Cette classe contient toutes les méthodes de l'interface avec une implémentation vide({}). Il suffit donc de créer un classe écouteur qui hérite de *MouseAdapter* et qui permet de redéfinir seulement la méthode qu'on va utiliser : *MouseClicked*

Étape 1 : créer une classe qui hérite de *MouseAdpater*

```
class EcouteurClasse extends MouseAdpater
{
    public void mouseClicked (MouseEvent ev) {
        int x = ev.getX() ; // ( retourne l'abscisse du curseur au moement du clic
        int y = ev.getY() ; // ( retourne l'ordonnée du curseur au moement du clic
        System.out.println ("clic au point de coordonnees " + x + ", " + y) ;
    };
}
```

Étape 2 : associer un écouteur de type *EcouteurClasse* à la fenêtre sur laquelle on va cliquer. Ceci se fait grâce à la méthode *addMouseListener* comme suit :

```
import javax.swing.* ;
import java.awt.event.*;
class MaFenetre extends JFrame implements MouseListener{
    public MaFenetre (){
        super ("Ma premiere fenetre") ;
        setSize (800, 120) ;
        addMouseListener(new EcouteurClasse());
    }
}
public class Premfen{
    public static void main (String args[]){
        MaFenetre fen = new MaFenetre();
        fen.setVisible (true);
    }
}
```

4ème méthode : créer une classe anonyme qui hérite de *MouseAdapter*

Étape 1 : créer une classe qui hérite de *MouseAdpater*

```
import javax.swing.* ;
import java.awt.event.*;
class MaFenetre extends JFrame implements MouseListener{
    public MaFenetre (){
        super ("Ma premiere fenetre") ;
        setSize (800, 120) ;
        addMouseListener(new MouseAdapter(){
            public void mouseClicked (MouseEvent ev) {
                int x = ev.getX() ;
                int y = ev.getY() ;
                System.out.println ("clic au point de coordonnees" +x+", "+ y) ;
            };
        });
    }
}
public class Premfen{
    public static void main (String args[]){
        MaFenetre fen = new MaFenetre();
        fen.setVisible (true);
    }
}
```

☐ **Avantage des 2 dernières méthdoes??**

Remarque : il existe une 5ème méthode qui utilise les classes internes mais qu'on ne va pas voir dans le cadre de ce cours.

II. Règle générale de l'utilisation des évènements

Soit une catégorie d'événements donnée *Xxx* (ex. *Mouse*). Si cette catégorie est associée à plusieurs méthodes (ex. *mouseClicked*, *mousePressed*), on pourra :

1ère méthode

1. Créer un écouteur qui sera un objet de la classe qui implémente l'interface *XxxListener* (la clause *implements XxxListener* devant figurer dans l'en-tête de classe de l'écouteur (ex. *implements MouseListener*)). Les méthodes dont on n'a pas besoin doivent être redéfinies mais peuvent avoir un corps vide (`{ }`). Toutes ces méthodes contiennent généralement un paramètre *XxxEvent* (ex. *MouseEvent*) qui correspond à l'évènement transmis à la méthode.

2. Associer cet écouteur à la source en utilisant la méthode *addXxxListener* (ex. *addMouseListener*)

2ème méthode

- Faire appel à une classe dérivée d'une classe adaptateur *XxxAdapter* (Ex. *MouseAdapter*) et ne fournir que les méthodes qui nous intéressent.

Remarque : lorsque la catégorie ne dispose que d'une seule méthode, Java ne prévoit pas généralement de classe adaptateur car elle n'aurait aucune utilité. Donc il faut utiliser la première solution.

L'objet écouteur pourra être n'importe quel objet; en particulier, il pourra s'agir de l'objet source lui-même (ex. la fenêtre est en même temps source et écouteur).

III. Gestion des évènements liés aux boutons : implémentation de l'interface ActionListener

Un bouton ne peut déclencher qu'un seul évènement lorsque l'utilisateur clique sur lui (ou le sélectionne et appuie sur Entrée).

La catégorie d'évènements liée aux boutons s'appelle *Action* et ne dispose que d'une seule méthode qui s'appelle *actionPerformed*. En suivant la 1ère méthode de la **Règle générale de l'utilisation des évènements** de la section II, l'utilisation de la catégorie *Action* avec un bouton se fait comme suit :

- Créer un écouteur qui sera un objet de la classe qui implémente l'interface *ActionListener* (la clause *implements ActionListener* devant figurer dans l'en-tête de classe de l'écouteur. La méthode *actionPerformed* doit être redéfinie. Cette méthode contient comme paramètre un objet de type *ActionEvent* (c'est l'évènement transmis à la méthode).

Exemple :

```
import javax.swing.* ;
import java.awt.event.*;
class MaFenetre extends JFrame implements ActionListener{
    public MaFenetre (){
        super ("Ma premiere fenetre") ;
        setSize (300, 150) ;
        Container c = getContentPane() ;
        JButton bValider ;
        bValider = new JButton ("valider") ;
        c.add(bValider);
        bValider.addActionListener(this);
    }
    public void actionPerformed (ActionEvent ev){
        System.out.println ("Vous avez appuyez sur le bouton Valider") ;
    }
}
public class Premfen{
    public static void main (String args[]){
        MaFenetre fen = new MaFenetre() ;
        fen.setVisible (true);
    }
}
```

Remarque : on ne peut pas utiliser la 2ème règle générale de l'utilisation des évènements car il n'existe pas une classe appelée *ActionAdapter*.

1) Gestion de plusieurs boutons

a) La fenêtre comme écouteur des boutons

Dans une fenêtre, on peut avoir plusieurs boutons. Si la fenêtre est l'écouteur de tous ces boutons, la question est comment savoir sur quel bouton, on a appuyé :

1ère solution : utiliser la méthode getSource() : cette méthode permet de retourner une référence sur l'objet qui a déclenché l'évènement *Action*.

Exemple :

```
import javax.swing.* ;
import java.awt.event.*;
import java.awt.*;
class MaFenetre extends JFrame implements ActionListener{
    private JButton bValider ;
    private JButton bAnnuler;
    public MaFenetre (){
        super ("Ma premiere fenetre") ;
        setSize (300, 150) ;
        Container c = getContentPane() ;
        c.setLayout(new FlowLayout());
        bValider = new JButton ("valider") ;
        c.add(bValider);
        bAnnuler = new JButton ("annuler") ;
        c.add(bAnnuler);
        bValider.addActionListener(this);
        bAnnuler.addActionListener(this);
    }
    public void actionPerformed (ActionEvent ev){
        if(ev.getSource()==bValider){
            System.out.println ("Vous avez appuyez sur le bouton Valider") ;
        }
        if(ev.getSource()==bAnnuler){
            System.out.println ("Vous avez appuyez sur le bouton Annuler") ;
        }
    }
}
public class Premfen{
    public static void main (String args[]){
        MaFenetre fen = new MaFenetre() ;
        fen.setVisible (true);
    }
}
```

2ème solution : utiliser la méthode getActionCommand() : cette méthode permet de retourner la chaîne de commande (sous forme d'une chaîne de caractères) associée au bouton qui a déclenché l'évènement .

Par défaut cette chaîne est l'étiquette du bouton. Par exemple pour le bouton bValider, cette chaîne est la chaîne "valider".

Exemple :

```
import javax.swing.* ;
import java.awt.event.*;
import java.awt.*;
class MaFenetre extends JFrame implements ActionListener{
    private JButton bValider ;
    private JButton bAnnuler;
    public MaFenetre (){
        super ("Ma premiere fenetre") ;
        setSize (300, 150) ;
        Container c = getContentPane() ;
        c.setLayout(new FlowLayout());
        bValider = new JButton ("valider") ;
        c.add(bValider);
        bAnnuler = new JButton ("annuler") ;
        c.add(bAnnuler);
        bValider.addActionListener(this);
        bAnnuler.addActionListener(this);
    }
}
```

```

}
public void actionPerformed (ActionEvent ev){
String nom= ev.getActionCommand();
if(nom.compareTo("valider")==0){
System.out.println ("Vous avez appuyez sur le bouton Valider") ;
}
if(nom.compareTo("annuler")==0){
System.out.println ("Vous avez appuyez sur le bouton Annuler") ;
}
}
}
}
public class Premfen{
public static void main (String args[]){
MaFenetre fen = new MaFenetre() ;
fen.setVisible (true);
}
}

```

Remarque 1 : on peut changer la chaîne de commande sans que l'étiquette change de nom en utilisant la méthode `setActionCommand()`.

Exemple :

```
bValider.setActionCommand("val");
```

Remarque 2 : la méthode `getSource` fonctionne avec tous les composants. Par contre, la méthode `getActionCommand` ne fonctionne qu'avec les composants générant un événement de la catégorie *Action*.

b) Ecouteur différent de la fenêtre pour tous les boutons

L'idée est de créer une classe qui implémente `ActionListener`. Le problème est qu'on n'a plus accès aux références des boutons (attributs privés de la classe).

Solution 1 :

```

import javax.swing.*.* ;
import java.awt.event.*;
import java.awt.*;
class Ecouteur implements ActionListener{
public void actionPerformed (ActionEvent ev){
String nom= ev.getActionCommand();
if(nom.compareTo("valider")==0){
System.out.println ("Vous avez appuyez sur le bouton Valider") ;
}
if(nom.compareTo("annuler")==0){
System.out.println ("Vous avez appuyez sur le bouton Annuler") ;
}
}
}
}
class MaFenetre extends JFrame {
private JButton bValider ;
private JButton bAnnuler;
public MaFenetre (){
super ("Ma premiere fenetre") ;
setSize (300, 150) ;
Container c = getContentPane() ;
c.setLayout(new FlowLayout());
bValider = new JButton ("valider") ;
c.add(bValider);
bAnnuler = new JButton ("annuler") ;
c.add(bAnnuler);
bValider.addActionListener(new Ecouteur());
bAnnuler.addActionListener(new Ecouteur());
}
}
public class Premfen{
public static void main (String args[]){
MaFenetre fen = new MaFenetre() ;
}
}

```

```
fen.setVisible (true);
}
}
```

Solution 2 :

```
import javax.swing.* ;
import java.awt.event.*;
import java.awt.*;
class Ecouteur implements ActionListener{
private int numero;
Ecouteur(int n){
numero=n;
}
public void actionPerformed (ActionEvent ev){
if(numero==1){
System.out.println ("Vous avez appuyez sur le bouton Valider") ;
}
if(numero==2){
System.out.println ("Vous avez appuyez sur le bouton Annuler") ;
}
}
}
class MaFenetre extends JFrame {
private JButton bValider ;
private JButton bAnnuler;
public MaFenetre (){
super ("Ma premiere fenetre") ;
setSize (300, 150) ;
Container c = getContentPane() ;
c.setLayout(new FlowLayout());
bValider = new JButton ("valider") ;
c.add(bValider);
bAnnuler = new JButton ("annuler") ;
c.add(bAnnuler);
bValider.addActionListener(new Ecouteur(1));
bAnnuler.addActionListener(new Ecouteur(2));
}
}
public class Premfen{
public static void main (String args[]){
MaFenetre fen = new MaFenetre() ;
fen.setVisible (true);
}
}
```

c) Ecouteur différent de la fenêtre et spécifique à chaque bouton

On pourra créer un écouteur pour chaque bouton

```
import java.awt.event.*;
import java.awt.*;
class EcouteurValider implements ActionListener{
public void actionPerformed (ActionEvent ev){
System.out.println ("Vous avez appuyez sur le bouton Valider") ;
}
}
```

```
class EcouteurAnnuler implements ActionListener{
public void actionPerformed (ActionEvent ev){
System.out.println ("Vous avez appuyez sur le bouton Annuler") ;
}
}
class MaFenetre extends JFrame {
private JButton bValider ;
private JButton bAnnuler;
public MaFenetre (){
super ("Ma premiere fenetre") ;
setSize (300, 150) ;
Container c = getContentPane() ;
```

```

c.setLayout(new FlowLayout());
bValider = new JButton ("valider") ;
c.add(bValider);
bAnnuler = new JButton ("annuler") ;
c.add(bAnnuler);
bValider.addActionListener(new EcouteurValider());
bAnnuler.addActionListener(new EcouteurAnnuler());
}
}
public class Premfen{
public static void main (String args[]){
MaFenetre fen = new MaFenetre() ;
fen.setVisible (true);
}
}

```

IV. Gestion des évènements les plus fréquents liés aux autres composants

Dans cette section, on présentera les évènements les plus fréquents associés aux différents composants

1) Les cases à cocher

Une action sur une case à cocher (le cocher ou le décocher) génère à chaque fois :

- un évènement *ActionEvent* de la catégorie *Action* : même type de fonctionnement qu'avec les boutons.
- un évènement *ItemEvent* de la catégorie *Item* : l'interface à implémenter est donc *ItemListener*. L'unique méthode de l'interface *ItemListener* est *itemStateChanged*.

```
public void itemStateChanged(ItemEvent ev)
```

Exemple :

```

import java.awt.* ;
import java.awt.event.* ;
import javax.swing.* ;
class MaFenetre extends JFrame implements ActionListener, ItemListener
{
private JCheckBox jCase1, jCase2;
public MaFenetre (){
super ("Ma premiere fenetre") ;
setSize (300, 150) ;
Container c = getContentPane() ;
c.setLayout(new FlowLayout());
jCase1 = new JCheckBox ("recevoir un e-mail") ;
c.add(jCase1) ;
jCase1.addItemListener (this) ;
jCase1.addActionListener (this) ;
jCase2= new JCheckBox ("recevoir un courrier postal") ;
jCase2.addActionListener (this) ;
jCase2.addItemListener (this) ;
c.add(jCase2) ;
}
public void itemStateChanged (ItemEvent ev)
{ Object src = ev.getSource() ;
if (src == jCase1) {
if(jCase1.isSelected()){
System.out.println ("Vous avez coché la case recevoir un e-mail ") ;
}
else{
System.out.println ("Vous avez décoché la case recevoir un e-mail ") ;
}
}
if (src == jCase2) {
if(jCase2.isSelected()){
System.out.println ("Vous avez coché la case recevoir un courrier postal ") ;
}
}
}
}

```

```

    }
    else{
        System.out.println ("Vous avez décoché la case recevoir un courrier postal ") ;
    }
}
}

public void actionPerformed (ActionEvent ev)
{
    Object src = ev.getSource() ;
    if (src == jCase1) {
        if(jCase1.isSelected()){
            System.out.println ("Vous avez coché la case recevoir un e-mail ") ;
        }
        else{
            System.out.println ("Vous avez décoché la case recevoir un e-mail ") ;
        }
    }
    if (src == jCase2) {
        if(jCase2.isSelected()){
            System.out.println ("Vous avez coché la case recevoir un courrier postal ") ;
        }
        else{
            System.out.println ("Vous avez décoché la case recevoir un courrier postal ") ;
        }
    }
}

public class Premfen
{ public static void main (String args[])
{ MaFenetre fen = new MaFenetre() ;
fen.setVisible(true) ;
}
}

```

Remarque: généralement, on traite un seul évènement : soit *ActionEvent* soit *ItemEvent*. Cependant, si dans l'action d'un bouton par exemple, on voudrait activer la sélection ou la désélection d'une case à cocher (ex. *jCase1.setSelected(true)*), seul l'évènement *ItemEvent* est déclenché mais pas *ActionEvent*

2) Les boutons radio

Soit un objet de type *ButtonGroup* composé de 2 boutons radio *r1* et *r2*. Une sélection sur un bouton *r1* par exemple (le cocher ou le décocher) génère à la fois :

- un évènement *ActionEvent* de la catégorie *Action* ou un évènement *ItemEvent* de la catégorie *Item* (voir case à cocher). Ces évènements sont déclenchés par le bouton radio sélectionné *r1*

- un évènement *ItemEvent* de la catégorie *Item* déclenché par le bouton radio *r2* puisque son état a changé lorsque *r1* a été sélectionné (il n'est plus sélectionné)

Exemple :

```

import java.awt.* ;
import java.awt.event.* ;
import javax.swing.* ;

class MaFenetre extends JFrame implements ActionListener, ItemListener
{
    private JRadioButton r1, r2;
    public MaFenetre (){
        super ("Ma premiere fenetre") ;
        setSize (300, 150) ;
        Container c = getContentPane() ;
        c.setLayout(new FlowLayout());
        ButtonGroup gr = new ButtonGroup() ;
        r1 = new JRadioButton ("envoyer", true) ;
        gr.add(r1) ;
        c.add(r1) ;
    }
}

```

```

r1.addItemListener (this);
r1.addActionListener (this);
r2 = new JRadioButton ("ne pas envoyer") ;
gr.add(r2) ;
c.add(r2) ;
r2.addItemListener (this) ;
r2.addActionListener (this) ;
}
public void itemStateChanged (ItemEvent ev)
{
Object source = ev.getSource() ;
if (source == r1) System.out.println ("changement r1") ;
if (source == r2) System.out.println ("changement r2") ;
System.out.println ("r1" + r1.isSelected() + ", r2 :" + r2.isSelected() ) ;
}
public void actionPerformed (ActionEvent ev)
{ Object source = ev.getSource() ;
if (source == r1) System.out.println ("selection r1") ;
if (source == r2) System.out.println ("selection r2") ;
System.out.println ("r1" + r1.isSelected() + ", r2 :" + r2.isSelected() ) ;
}
}
public class Premfen
{ public static void main (String args[])
{ MaFenetre fen = new MaFenetre() ;
fen.setVisible(true) ;
}
}

```

☐ Si on clique sur ne pas envoyer, on aura l'affichage suivant :

```

changement r1
r1 : false, r2 : false
changement r2
r1 : false, r2 : true
selection r2
r1 : false, r2 : true

```

3) Les champs de texte

Les évènements les plus fréquents associés à un champ de texte :

- un évènement *ActionEvent* lorsque l'utilisateur appuie sur Entrée après avoir taper son texte
- un évènement *FocusEvent* fassocié au focus. Pour le traiter, on doit
 - soit implémenter l'interface *FocusListener* qui contient les méthodes

```

public void focusGained(FocusEvent e) // lorsque le champ a le focus
public void focusLost(FocusEvent e) // lorsque le champ a perdu le focus

```

- soit utiliser l'adaptateur *FocusAdapter* (voir règle générale)

4) Les Boites de liste

L'évènement qui est déclenché avec une liste est l'évènement *ListSelectionEvent* de la catégorie *ListSelection*. L'interface *ListSelectionListener* contient une seule méthode qui est :

```

public void valueChanged(ListSelectionEvent e)

```

Cet évènement est déclenché :

- lors de l'appui sur le bouton de la souris sur une valeur (car il suppose qu'une autre valeur a été désélectionnée)
- lors du relâchement du bouton de la souris (car la nouvelle valeur vient d'être sélectionnée)

☐ Pour ne pas traiter deux fois la même chose (lors de l'appui et du relâchement du bouton), on pourra écrire :


```

public void valueChanged (ListSelectionEvent e)
{ if (!e.getValueIsAdjusting())
{ // faire le traitement car on est sûr qu'on a relâché le bouton de la souris
}
}

```

Exemple :

```

import java.awt.* ;
import java.awt.event.* ;
import javax.swing.* ;
import javax.swing.event.* ; // utile pour ListSelectionListener
class MaFenetre extends JFrame implements ListSelectionListener
{ public MaFenetre ()
{ setTitle ("MaFenetre") ;
setSize (300, 300) ;
Container c = getContentPane() ;
c.setLayout (new FlowLayout() ) ;
liste = new JList (jours) ;
c.add(liste) ;
liste.addListSelectionListener (this) ;
}
public void valueChanged (ListSelectionEvent e)
{ if (!e.getValueIsAdjusting())
{ System.out.println ("Valeurs selectionnees :)") ;
Object[] valeurs = liste.getSelectedValues() ;
for (int i = 0 ; i<valeurs.length ; i++)
System.out.println ((String) valeurs[i]) ;
}
}
private String[] jours = {"Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi", "Samedi", "Dimanche" } ;
private JList liste ;
}
public class Premfen
{ public static void main (String args[])
{ MaFenetre fen = new MaFenetre () ;
fen.setVisible(true) ;
}
}

```

5) Les boîtes combo

Lors d'une sélection d'une valeur dans un combo, deux évènements sont déclenchés: *ItemEvent* et *ActionEvent*

- Pour chaque sélection d'une valeur, l'évènement *ItemEvent* est déclenché deux fois :

- une fois à cause de la valeur qui a été désélectionnée
- une fois à cause de la nouvelle valeur sélectionnée

☐ Si la même valeur est sélectionnée, pas de déclenchement d'évènement *ItemEvent*

- Pour chaque sélection, l'évènement *actionEvent* est déclenché une fois même si c'est la même valeur qui est sélectionnée

Remarque : si la boîte combo est éditable, les événements *ItemEvent* et *ActionEvent* sont aussi déclenchés après la validation par une *Entrée* de la nouvelle valeur saisie.

V. Gestion des événements de bas niveau

1. Événements liés à la souris :

a) Les méthodes `mousePressed` et `mouseReleased`

- **`mousePressed`** : est appelée à chaque appui d'un bouton de la souris
- **`mouseReleased`** : est appelée à chaque relâchement d'un bouton de la souris
- **`mouseClicked`** : est appelée à chaque clic "complet" d'un bouton (appui puis relâchement) à condition que la souris ne s'est pas déplacée.

b) Identification du bouton (droit, gauche ou celui du milieu)

Pour savoir quel type de bouton a été appuyé (gauche, droit ou centre), la classe `InputEvent` contient 3 constantes

- `InputEvent.BUTTON1_MASK` associé au bouton gauche
- `InputEvent.BUTTON2_MASK` associé au bouton central
- `InputEvent.BUTTON3_MASK` associé au bouton droit

Pour tester si le bouton droit a été appuyé, on fait

```
if ((e.getModifiers() & InputEvent.BUTTON3_MASK) != 0) ...
```

Pour le bouton droit, on pourra aussi utiliser la méthode `isPopupTrigger` de la classe `MouseEvent`.

Mais il faudra l'utiliser pour le type d'évènement `mouseReleased` et non pas `mouseClicked`

```
public void mouseReleased(MouseEvent e){  
    if(e.isPopupTrigger()==true){//□ bouton droit appuyé  
    ...  
    }  
}
```

Remarque : pour savoir le nombre de clics, on utilise la méthode `getClickCount()` de la classe `MouseEvent` et qui retourne le nombre de cliques successifs en un même point c-à-d tant qu'on n'a pas bougé la souris. Dès qu'on bouge la souris ou qu'un temps est écoulé entre un clic et un autre, ce compteur est remis à 0.

c) Déplacements de souris

• les méthodes `mouseEntered` et `mouseExited`

□ **`mouseEntered`** : ce type d'évènement est déclenché lorsque la souris se déplace de l'extérieur vers l'intérieur du composant qui a comme écouteur un objet implémentant `MouseListener`

□ **`mouseExited`** : ce type d'évènement est déclenché lorsque la souris se déplace de l'intérieur vers l'extérieur du composant qui a comme écouteur un objet implémentant `MouseListener`

• les méthodes `mouseMoved` et `mouseDragged` de l'interface `MouseMotionListener`

Les événements de type `mouseMoved` et `mouseDragged` sont déclenchés par un composant lorsque lorsque le curseur se déplace sur le composant :

- **`mouseMoved`** : le curseur se déplace sur le composant sans qu'aucun bouton ne soit appuyé
- **`mouseDragged`** : le curseur se déplace sur le composant tout en ayant un bouton appuyé pendant le déplacement.

Remarque : ces deux méthodes appartiennent à l'interface `MouseMotionListener` :

```
public void mouseMoved(MouseEvent e){... }  
public void mouseDragged(MouseEvent e){... }
```

2) Evènements liés au clavier

Les méthodes de l'interface KeyListener et associée à la catégorie d'évènement Key sont :

- keyPressed, appelée lorsqu'une touche a été enfoncée,
- keyReleased, appelée lorsqu'une touche a été relâchée,
- keyTyped, appelée (en plus des deux précédentes), lorsque un caractère Unicode a été généré (Ex. pas d'appel de cette méthode si on appuie seulement sur shift)

Question : donnez la suite des méthodes de l'interface KeyListener appelée pour saisir le caractère A?

- keyPressed quand on appuie sur Shift,
- keyPressed quand on appuie sur a,
- keyTyped pour le caractère A.
- keyReleased quand on relâche sur a,
- keyReleased quand on relâche sur Shift,

a. Identification des touches

La classe KeyEvent contient 3 méthodes statiques importantes :

- getKeyChar(): retourne le caractère saisi (n'affichent pas les caractères shift, Alt , ctrl, ...)
- getKeyCode() : retourne un code pour chaque touche.

Exemple :

- VK_0 : pour le 0 du pavé alphabétique
- VK_1 : pour le 0 du pavé alphabétique
- ...
- VK_NUMPAD0 : pour le 0 du pavé numérique
- ...
- VK_A : pour la caractère A (même pour le caractère a)
- ...
- VK_Z : pour la caractère Z (même pour le caractère z)
- VK_F1 à VK_F24 touches fonction F1 à F24
- VK_ALT touche Alt
- VK_ALT_GRAPH touche Alt graphique
- VK_CAPS_LOCK touche verrouillage majuscules
- VK_CONTROL touche Ctrl
- VK_DELETE touche Suppr
- VK_DOWN touche flèche bas
- VK_END touche Fin
- VK_ENTER touche de validation
- VK_ESCAPE touche Echap
- VK_HOME touche Home
- VK_INSERT touche Insert
- VK_LEFT touche flèche gauche
- VK_NUM_LOCK touche verrouillage numérique
- VK_PAGE_DOWN touche Page suivante
- VK_PAGE_UP touche Page précédente
- VK_PRINTSCREEN touche Impression écran
- VK_RIGHT touche flèche droite
- VK_SCROLL_LOCK touche arrêt défilement
- VK_SHIFT touche majuscules temporaire
- VK_SPACE touche espace
- VK_TAB touche de tabulation

Remarque : même si la touche 5 du pavé alphanumérique contient aussi (et [, VK_5 correspondra toujours à la valeur 5 quel que soit le caractère tapé.

- getKeyText(int code) : retourne le texte associé au code retourné par getKeyCode()

Ex.

```
public void keyPressed (KeyEvent e){// on suppose qu'on appuie sur Ctrl
int code = e.getKeyCode() ; □ retourne l'entier correspondant à VK_CONTROL
System.out.println (e.getKeyText (code)) ; □ affiche Ctrl
}
```

b. Test sur certaines touches

KeyEvent dispose d'autres méthodes pour savoir si certaines touches spécifiques ont été appuyées ou non :

- isAltDown() : retourne true si Alt est appuyé
- isAltGraphDown : retourne true si Alt Graphique est appuyé
- isControlDown : retourne true si Ctrl est appuyé
- isShiftDown : retourne true si Shift est appuyé
- isMetaDown : retourne true si Alt ou Alt Graphique ou Ctrl ou Shift est appuyé

3) Evènements liés aux fenêtres

Ex.

```
import javax.swing.* ;
import java.awt.* ;
import java.awt.event.* ;
class MaFenetre extends JFrame implements WindowListener
{
    public MaFenetre ()
    {
        setTitle ("Evenements fenetre") ; setSize (300, 100) ;
        addWindowListener (this) ;
    }
    public void windowClosing (WindowEvent e)
    {
        System.out.println ("fenetre en cours fermeture") ;
    }
    public void windowOpened (WindowEvent e)
    {
        System.out.println ("ouverture fenetre") ;
    }
    public void windowIconified (WindowEvent e)
    {
        System.out.println ("fenetre en icone") ;
    }
    public void windowDeiconified(WindowEvent e)
    {
        System.out.println ("icone en fenetre") ;}

    public void windowClosed (WindowEvent e)
    {
        System.out.println ("fenetre fermee") ;
    }
}
```

```
}  
public void windowActivated (WindowEvent e)  
{  
    System.out.println ("fenetre activee") ;  
}  
public void windowDeactivated (WindowEvent e)  
{  
    System.out.println ("fenetre desactivee") ;  
}  
}
```

☐ **Affichage**

fenetre activee

ouverture fenetre

fenetre en icone

fenetre desactivee

fenetre activee

icone en fenetre

fenetre activee

fenetre en cours fermeture

fenetre desactivee

☐ L'évènement `windowClosed` est appelé si on avait `setDefaultCloseOperation(DISPOSE_ON_CLOSE)`;